LAPORAN

WORKSHOP Mobile Web Server

(TIF310808)

Acara 13

SEMESTER



Nama Tugas

SERVICE LAYER, VALIDASI, SESSION, DAN BUSINESS LOGIC

Nama:

Adam Ridho Saifulloh

NIM:

E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Pada pengembangan aplikasi web yang semakin kompleks, Controller dapat menjadi terlalu panjang apabila validasi, aturan bisnis, dan akses database diletakkan dalam satu class. Acara 13 menerapkan Service Layer untuk memisahkan logika bisnis dari Controller dan Repository. Selain itu, digunakan validasi data dan Flash Message berbasis session agar proses tambah dan ubah data mahasiswa lebih terstruktur dan memberikan informasi yang jelas kepada pengguna.

1.2 Tujuan

- Menerapkan Service Layer pada aplikasi Sistem Informasi Akademik.
- Memindahkan validasi dan business logic dari Controller ke MahasiswaService.
- Memisahkan akses database melalui MahasiswaRepository dan ProdiRepository.
- Menerapkan Flash Message menggunakan session.
- Menguji proses tambah, ubah, hapus, validasi, dan pencarian data mahasiswa.

1.3 Pokok Bahasan

Service Layer, Validasi, Session, dan Business Logic.

BAB II ALAT DAN BAHAN

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon
- PHP
- HTML, CSS, Bootstrap
- Modul Praktikum

BAB III PROSEDUR KERJA

Prosedur berikut disusun mengikuti langkah praktikum Acara 13 pada BKPM dan disesuaikan dengan source code pada ZIP Bkpm_Acara_13 yang diberikan.

1. Menyiapkan project

Project Sistem Informasi Akademik dari praktikum sebelumnya digunakan sebagai dasar. Project Acara 13 memiliki folder app, config, database, dan public. Struktur aplikasi tetap memanfaatkan MVC dan Repository Pattern yang telah dibuat pada praktikum sebelumnya.

2. Membuat MahasiswaService

Dibuat class MahasiswaService pada app/Services/MahasiswaService.php. Class ini menjadi tempat utama untuk mengatur business logic mahasiswa. Service menerima MahasiswaRepository dan ProdiRepository melalui constructor injection.

3. Menghubungkan Service dengan Repository
MahasiswaRepository digunakan untuk operasi data mahasiswa, sedangkan ProdiRepository digunakan untuk mengambil daftar program studi dan memeriksa keberadaan prodi. Dengan pemisahan ini, Service tidak menjalankan SQL secara langsung.

4. Memindahkan validasi ke Service
Validasi input dipindahkan dari Controller ke method validate() pada MahasiswaService. Data NIM, nama, email, prodi, dan angkatan diproses menggunakan setter pada model Mahasiswa. Service juga memeriksa NIM duplikat melalui existsByNim() dan memeriksa prodi melalui exists().

5. Membuat proses create()
Method create() pada MahasiswaService menerima input form, menjalankan validasi, kemudian memanggil MahasiswaRepository->create() jika data valid. Method mengembalikan hasil dalam bentuk array yang berisi status keberhasilan, pesan, error, dan objek Mahasiswa.

6. Membuat proses update()
Method update() melakukan proses yang sama seperti create(), tetapi menerima ID mahasiswa. Service memvalidasi data kemudian memanggil MahasiswaRepository->update() jika valid.

7. Menerapkan Flash Message
Flash Message dibuat melalui helper setFlash() dan getFlash(). Pesan disimpan pada $_SESSION['flash'], ditampilkan satu kali pada header.php, lalu dihapus kembali dari session. Pesan dapat berupa keberhasilan maupun kegagalan proses.

8. Menyederhanakan MahasiswaController
MahasiswaController tidak lagi berisi validasi dan query database. Controller hanya menerima request, memanggil method Service, lalu menentukan response berupa redirect atau menampilkan form beserta pesan error.

9. Merakit objek pada public/index.php
Pada entry point public/index.php, objek Database dibuat terlebih dahulu, kemudian digunakan untuk membuat MahasiswaRepository dan ProdiRepository. Kedua repository diberikan kepada MahasiswaService, kemudian Service diberikan kepada MahasiswaController.

10. Menguji aplikasi
Aplikasi diuji melalui browser untuk memastikan proses tampil data, pencarian, tambah data, ubah data, hapus data, validasi NIM, validasi email, validasi prodi, serta Flash Message berjalan sesuai kebutuhan praktikum.

3.1 Alur Proses Aplikasi
Alur pengolahan data pada Acara 13 adalah: Browser → Controller → Service → Repository → Database. Controller menerima request dan menentukan response, Service

menjalankan validasi serta business logic, Repository menjalankan query database, sedangkan Database menyediakan koneksi PDO.

BAB IV SOURCE CODE

Source code pada bagian ini diambil langsung dari ZIP Bkpm_Acara_13 yang diberikan. Kode ditampilkan sebagai dokumentasi implementasi sesuai file project.

app/Services/MahasiswaService.php

```php
<?php

require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';
require_once __DIR__ . '/../Repositories/ProdiRepository.php';

// Service Layer: tempat logika bisnis mahasiswa.
//   Controller  -> mengatur alur (terima request, tentukan response)
//   Service     -> validasi + aturan bisnis (class ini)
//   Repository  -> menjalankan query ke database
class MahasiswaService
{
    // Constructor injection: kedua Repository diberikan dari luar (lihat
public/index.php)
    public function __construct(
        private MahasiswaRepository $repo,
        private ProdiRepository $prodiRepo
    ) {}

    // ---------- Proses baca data (diteruskan ke Repository) ----------

    public function all(string $keyword = ''): array
    {
        return $this->repo->all($keyword);
    }

    public function find(int $id): ?Mahasiswa
    {
        return $this->repo->find($id);
    }

    public function daftarProdi(): array
    {
        return $this->prodiRepo->all();
    }

    // ---------- Proses tulis data ----------
    // Semua method di bawah mengembalikan array:
    //   success (bool), message (string, untuk flash message),
```

```php
    //   errors (array per field), data (objek Mahasiswa, dipakai untuk
mengisi ulang form)

    public function create(array $input): array
    {
        [$m, $errors] = $this->validate($input);

        if (!empty($errors)) {
            return $this->gagal($m, $errors);
        }

        try {
            $this->repo->create($m);
        } catch (PDOException $e) {
            return $this->gagal($m, $this->errorDatabase($e));
        }

        return [
            'success' => true,
            'message' => 'Data mahasiswa berhasil ditambahkan.',
            'errors'  => [],
            'data'    => $m,
        ];
    }

    public function update(int $id, array $input): array
    {
        [$m, $errors] = $this->validate($input, $id);

        if (!empty($errors)) {
            return $this->gagal($m, $errors);
        }

        try {
            $this->repo->update($m);
        } catch (PDOException $e) {
            return $this->gagal($m, $this->errorDatabase($e));
        }

        return [
            'success' => true,
            'message' => 'Data mahasiswa berhasil diubah.',
            'errors'  => [],
            'data'    => $m,
        ];
    }

    public function delete(int $id): array
    {
```

```php
        if ($this->repo->find($id) === null) {
            return ['success' => false, 'message' => 'Data mahasiswa tidak
ditemukan.', 'errors' => [], 'data' => null];
        }

        try {
            $this->repo->delete($id);
        } catch (PDOException $e) {
            return ['success' => false, 'message' => 'Data gagal dihapus.',
'errors' => [], 'data' => null];
        }

        return ['success' => true, 'message' => 'Data mahasiswa berhasil
dihapus.', 'errors' => [], 'data' => null];
    }

    // ---------- Validasi (dipindah dari Controller) ----------

    // Mengembalikan [objek Mahasiswa, array error].
    // $id = 0 untuk data baru, > 0 untuk data yang sedang diubah.
    private function validate(array $input, int $id = 0): array
    {
        $m = new Mahasiswa();
        $m->setId($id);
        $errors = [];

        // 1. Aturan format: dicek lewat setter pada class Mahasiswa
        $fields = [
            'nim'      => ['setNim',      $input['nim'] ?? ''],
            'nama'     => ['setNama',     $input['nama'] ?? ''],
            'email'    => ['setEmail',    $input['email'] ?? ''],
            'prodi_id' => ['setProdiId',  (int) ($input['prodi_id'] ?? 0)],
            'angkatan' => ['setAngkatan', (int) ($input['angkatan'] ?? 0)],
        ];

        foreach ($fields as $field => [$setter, $nilai]) {
            try {
                $m->$setter($nilai);
            } catch (InvalidArgumentException $e) {
                $errors[$field] = $e->getMessage();
            }
        }

        // 2. Aturan bisnis: butuh data dari database, jadi dicek di Service
        if (!isset($errors['nim']) && $this->repo->existsByNim($m->getNim(),
$id)) {
            $errors['nim'] = 'NIM sudah terdaftar.';
        }
```

```
        if (!isset($errors['prodi_id']) && !$this->prodiRepo->exists($m->getProdiId())) {
            $errors['prodi_id'] = 'Program studi tidak tersedia.';
        }

        return [$m, $errors];
    }

    // Susun hasil gagal. Pesan flash dibedakan agar pengguna tahu penyebabnya.
    private function gagal(Mahasiswa $m, array $errors): array
    {
        $message = 'Data gagal disimpan. Periksa kembali isian form.';

        if (isset($errors['nim']) && $errors['nim'] === 'NIM sudah terdaftar.') {
            $message = 'Data gagal disimpan. NIM sudah terdaftar.';
        }

        return ['success' => false, 'message' => $message, 'errors' => $errors, 'data' => $m];
    }

    // Jaring pengaman: NIM duplikat yang lolos pengecekan (kolom UNIQUE) -> error 23000
    private function errorDatabase(PDOException $e): array
    {
        if ($e->getCode() === '23000') {
            return ['nim' => 'NIM sudah terdaftar.'];
        }
        throw $e;
    }
}
```

app/Repositories/MahasiswaRepository.php

```
<?php

require_once __DIR__ . '/../Models/BaseModel.php';
require_once __DIR__ . '/../Models/Mahasiswa.php';

// Repository Pattern: seluruh query SQL mahasiswa ada di class ini.
// Mewarisi BaseModel, jadi $this->pdo sudah tersedia dari class induk
// (Database disuntikkan lewat constructor BaseModel).
class MahasiswaRepository extends BaseModel
{
    // Semua mahasiswa + nama prodi (JOIN). Jika $keyword diisi, cari nama atau NIM dengan LIKE
    public function all(string $keyword = ''): array
```

```
    {
        $sql = "SELECT m.*, p.nama AS prodi_nama
                FROM mahasiswa m
                JOIN prodi p ON m.prodi_id = p.id";
        $params = [];

        if ($keyword !== '') {
            $sql .= " WHERE m.nama LIKE :nama OR m.nim LIKE :nim";
            $params['nama'] = '%' . $keyword . '%';
            $params['nim']  = '%' . $keyword . '%';
        }

        $sql .= " ORDER BY m.nim";

        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);

        $hasil = [];
        foreach ($stmt->fetchAll() as $row) {
            $hasil[] = Mahasiswa::dariBaris($row);
        }
        return $hasil;
    }

    public function find(int $id): ?Mahasiswa
    {
        $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id =
:id");
        $stmt->execute(['id' => $id]);
        $row = $stmt->fetch();

        return $row ? Mahasiswa::dariBaris($row) : null;
    }

    public function create(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan)
             VALUES (:nim, :nama, :email, :prodi_id, :angkatan)"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
        ]);
    }
```

```
    public function update(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "UPDATE mahasiswa
             SET nim = :nim, nama = :nama, email = :email, prodi_id =
:prodi_id, angkatan = :angkatan
             WHERE id = :id"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
            'id'       => $m->getId(),
        ]);
    }

    public function delete(int $id): void
    {
        $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
        $stmt->execute(['id' => $id]);
    }

    // Cek apakah NIM sudah dipakai mahasiswa lain.
    // $exceptId diisi saat edit, supaya NIM milik data itu sendiri tidak
dianggap duplikat.
    public function existsByNim(string $nim, int $exceptId = 0): bool
    {
        $stmt = $this->pdo->prepare("SELECT 1 FROM mahasiswa WHERE nim =
:nim AND id <> :id");
        $stmt->execute(['nim' => $nim, 'id' => $exceptId]);

        return (bool) $stmt->fetchColumn();
    }
}
```

app/Controllers/BaseController.php

```
<?php

// Induk semua Controller (Inheritance).
// Method yang dipakai bersama (view & redirect) ditulis sekali di sini,
// lalu diwarisi oleh controller lain lewat "extends BaseController".
abstract class BaseController
{
    // Tampilkan file view. Isi $data menjadi variabel di view,
    // contoh: ['daftar' => $x] -> di view bisa langsung memakai $daftar
    protected function view(string $view, array $data = []): void
```

```
    {
        extract($data);
        require __DIR__ . '/../Views/' . $view . '.php';
    }

    // Pindah halaman, pesan (opsional) disimpan sebagai flash message di
session
    protected function redirect(string $path, string $message = '', string
$type = 'success'): void
    {
        if ($message !== '') {
            setFlash($type, $message);
        }
        header('Location: ' . url($path));
        exit;
    }
}
```

app/Models/BaseModel.php

```
<?php

require_once __DIR__ . '/../Database.php';

// Induk semua class yang butuh akses database (Inheritance).
// Koneksi PDO diambil dari objek Database yang disuntikkan lewat
constructor,
// sehingga class turunan cukup memakai $this->pdo.
abstract class BaseModel
{
    protected PDO $pdo;

    public function __construct(Database $db)
    {
        $this->pdo = $db->getConnection();
    }
}
```

app/Models/Mahasiswa.php

```
<?php

class Mahasiswa
{
    // Semua atribut private: hanya bisa diakses lewat getter dan setter
    private int $id = 0;
    private string $nim = '';
    private string $nama = '';
    private string $email = '';
```

```php
    private int $prodiId = 0;
    private int $angkatan;
    private string $prodiNama = '';   // hanya untuk tampilan (hasil JOIN)

    public function __construct()
    {
        $this->angkatan = (int) date('Y');
    }

    // Buat objek dari satu baris data database (data dari database dianggap
sudah valid)
    public static function dariBaris(array $row): self
    {
        $m = new self();
        $m->id        = (int) $row['id'];
        $m->nim       = $row['nim'];
        $m->nama      = $row['nama'];
        $m->email     = $row['email'];
        $m->prodiId   = (int) $row['prodi_id'];
        $m->angkatan  = (int) $row['angkatan'];
        $m->prodiNama = $row['prodi_nama'] ?? '';
        return $m;
    }

    // ---------- Getter ----------
    public function getId(): int          { return $this->id; }
    public function getNim(): string      { return $this->nim; }
    public function getNama(): string     { return $this->nama; }
    public function getEmail(): string    { return $this->email; }
    public function getProdiId(): int     { return $this->prodiId; }
    public function getAngkatan(): int    { return $this->angkatan; }
    public function getProdiNama(): string { return $this->prodiNama; }

    // ---------- Setter (dengan validasi sederhana) ----------
    public function setId(int $id): void
    {
        $this->id = $id;
    }

    public function setNim(string $nim): void
    {
        $nim = trim($nim);

        if ($nim === '') {
            throw new InvalidArgumentException('NIM wajib diisi.');
        }
        if (!ctype_digit($nim)) {
            throw new InvalidArgumentException('NIM harus berupa angka.');
        }
```

```
        if (strlen($nim) > 20) {
            throw new InvalidArgumentException('NIM maksimal 20 digit.');
        }

        $this->nim = $nim;
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === '') {
            throw new InvalidArgumentException('Nama tidak boleh kosong.');
        }
        if (strlen($nama) > 100) {
            throw new InvalidArgumentException('Nama maksimal 100
karakter.');
        }

        $this->nama = $nama;
    }

    public function setEmail(string $email): void
    {
        $email = trim($email);

        if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException('Format email tidak valid.');
        }

        $this->email = $email;
    }

    public function setProdiId(int $prodiId): void
    {
        if ($prodiId <= 0) {
            throw new InvalidArgumentException('Pilih program studi.');
        }

        $this->prodiId = $prodiId;
    }

    public function setAngkatan(int $angkatan): void
    {
        if ($angkatan < 1990 || $angkatan > (int) date('Y') + 1) {
            throw new InvalidArgumentException('Angkatan tidak valid.');
        }

        $this->angkatan = $angkatan;
```

```
    }
}
```

app/Controllers/AuthController.php

```php
<?php

require_once __DIR__ . '/BaseController.php';

class AuthController extends BaseController
{
    public function loginForm(): void
    {
        // Kalau sudah login, tidak perlu lihat form login lagi
        if (!empty($_SESSION['logged_in'])) {
            $this->redirect('/mahasiswa');
        }

        $this->view('auth/login');
    }

    public function login(): void
    {
        $username = $_POST['username'] ?? '';
        $password = $_POST['password'] ?? '';

        // Login sementara menggunakan data tetap (hardcode)
        if ($username === 'admin' && $password === '12345') {
            session_regenerate_id(true);
            $_SESSION['logged_in'] = true;
            $_SESSION['user_name'] = 'Admin';

            $this->redirect('/mahasiswa', 'Selamat datang, Admin.');
        }

        $this->redirect('/login', 'Username atau password salah.',
'danger');
    }

    public function logout(): void
    {
        unset($_SESSION['logged_in'], $_SESSION['user_name']);

        $this->redirect('/login', 'Anda telah logout.', 'info');
    }
}
```

public/index.php

```php
<?php

session_start();

// BASE_PATH otomatis (contoh: /Bkpm_Acara_13/public), jadi tidak perlu
diubah manual
define('BASE_PATH', rtrim(str_replace('\\', '/',
dirname($_SERVER['SCRIPT_NAME'])), '/'));

require_once __DIR__ . '/../app/helpers.php';
require_once __DIR__ . '/../app/Controllers/AuthController.php';
require_once __DIR__ . '/../app/Controllers/MahasiswaController.php';

// Ambil path tanpa BASE_PATH, contoh: /mahasiswa/edit
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$uri = substr($uri, strlen(BASE_PATH));
$uri = '/' . trim($uri, '/');

// Login & logout (boleh diakses tanpa login)
if ($uri === '/login') {
    $auth = new AuthController();
    $_SERVER['REQUEST_METHOD'] === 'POST' ? $auth->login() : $auth-
>loginForm();
    exit;
}

if ($uri === '/logout') {
    (new AuthController())->logout();
}

// Semua halaman di bawah ini wajib login
cekLogin();

if ($uri === '/') {
    redirect('/mahasiswa');
}

// Merakit objek (Dependency Injection):
// Database -> Repository -> Service -> Controller
$database    = new Database();
$mhsRepo     = new MahasiswaRepository($database);
$prodiRepo   = new ProdiRepository($database);
$service     = new MahasiswaService($mhsRepo, $prodiRepo);
$controller  = new MahasiswaController($service);

// GET untuk menampilkan, POST untuk mengubah data
$parts  = explode('/', trim($uri, '/'));
$action = $parts[1] ?? 'index';
```

```
$allowed = ($_SERVER['REQUEST_METHOD'] === 'POST')
    ? ['store', 'update', 'destroy']
    : ['index', 'create', 'edit'];

if ($parts[0] === 'mahasiswa' && count($parts) <= 2 && in_array($action,
$allowed, true)) {
    $controller->$action();
} else {
    http_response_code(404);
    echo '<h1>404</h1><p>Halaman tidak ditemukan.</p>';
}
```

config/database.php

```
<?php

return [
    'host'     => 'localhost',
    'dbname'   => 'bkpm_acara_7',
    'username' => 'root',
    'password' => '',
    'charset'  => 'utf8mb4',
];
```

app/Views/header.php

```
<?php
$flash = getFlash();   // flash message: tampil sekali lalu hilang
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>SI Akademik</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body class="bg-light">

<nav class="navbar navbar-expand navbar-dark bg-dark mb-4">
    <div class="container">
        <span class="navbar-brand">SI Akademik</span>
        <div class="navbar-nav">
            <a class="nav-link" href="<?= url('/mahasiswa')
?>">Mahasiswa</a>
        </div>
        <div class="ms-auto d-flex align-items-center">
```

```
            <span class="navbar-text me-3"><?= e($_SESSION['user_name'] ??
'') ?></span>
            <a class="btn btn-outline-light btn-sm" href="<?= url('/logout')
?>">Logout</a>
        </div>
    </div>
</nav>

<main class="container pb-5">
    <?php if ($flash): ?>
        <div class="alert alert-<?= e($flash['type']) ?>"><?=
e($flash['message']) ?></div>
    <?php endif; ?>
```

app/Views/mahasiswa/form.php

```
<?php
// Variabel: $m (objek Mahasiswa), $action, $prodiList, $errors, $old (input
sebelumnya jika ada error)
$invalid = fn(string $f) => isset($errors[$f]) ? 'is-invalid' : '';
require __DIR__ . '/../header.php';
?>

<h1 class="h3 mb-3"><?= $m->getId() ? 'Edit' : 'Tambah' ?> Mahasiswa</h1>

<form method="POST" action="<?= e($action) ?>" class="card card-body"
novalidate>
    <input type="hidden" name="id" value="<?= $m->getId() ?>">

    <div class="mb-3">
        <label class="form-label">NIM</label>
        <input type="text" name="nim" class="form-control <?=
$invalid('nim') ?>"
               value="<?= e($old['nim'] ?? $m->getNim()) ?>">
        <div class="invalid-feedback"><?= e($errors['nim'] ?? '') ?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Nama</label>
        <input type="text" name="nama" class="form-control <?=
$invalid('nama') ?>"
               value="<?= e($old['nama'] ?? $m->getNama()) ?>">
        <div class="invalid-feedback"><?= e($errors['nama'] ?? '') ?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Email</label>
        <input type="text" name="email" class="form-control <?=
$invalid('email') ?>"
```

```
               value="<?= e($old['email'] ?? $m->getEmail()) ?>">
        <div class="invalid-feedback"><?= e($errors['email'] ?? '') ?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Program Studi</label>
        <?php $prodiTerpilih = (int) ($old['prodi_id'] ?? $m->getProdiId());
?>
        <select name="prodi_id" class="form-select <?= $invalid('prodi_id')
?>">
            <option value="">-- Pilih Prodi --</option>
            <?php foreach ($prodiList as $p): ?>
                <option value="<?= (int) $p['id'] ?>" <?= $prodiTerpilih ===
(int) $p['id'] ? 'selected' : '' ?>>
                    <?= e($p['kode'] . ' - ' . $p['nama']) ?>
                </option>
            <?php endforeach; ?>
        </select>
        <div class="invalid-feedback"><?= e($errors['prodi_id'] ?? '')
?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Angkatan</label>
        <input type="number" name="angkatan" class="form-control <?=
$invalid('angkatan') ?>"
               value="<?= e($old['angkatan'] ?? $m->getAngkatan()) ?>">
        <div class="invalid-feedback"><?= e($errors['angkatan'] ?? '')
?></div>
    </div>

    <div>
        <button type="submit" class="btn btn-primary">Simpan</button>
        <a href="<?= url('/mahasiswa') ?>" class="btn btn-
secondary">Batal</a>
    </div>
</form>

<?php require __DIR__ . '/../footer.php'; ?>
```

app/Views/mahasiswa/index.php

<?php require __DIR__ . '/../header.php'; ?>

```
<?php require __DIR__ . '/../header.php'; ?>

<div class="d-flex justify-content-between mb-3">
    <h1 class="h3">Data Mahasiswa</h1>
    <a href="<?= url('/mahasiswa/create') ?>" class="btn btn-primary">+
Tambah Mahasiswa</a>
```

```
</div>

<form method="GET" action="<?= url('/mahasiswa') ?>" class="row g-2 mb-3">
    <div class="col-md-6">
        <input type="text" name="q" value="<?= e($keyword) ?>" class="form-
control"
               placeholder="Cari berdasarkan nama atau NIM...">
    </div>
    <div class="col-auto">
        <button type="submit" class="btn btn-outline-primary">Cari</button>
        <?php if ($keyword !== ''): ?>
            <a href="<?= url('/mahasiswa') ?>" class="btn btn-outline-
secondary">Reset</a>
        <?php endif; ?>
    </div>
</form>

<table class="table table-striped table-bordered bg-white">
    <thead class="table-dark">
        <tr><th>No</th><th>NIM</th><th>Nama</th><th>Email</th><th>Prodi</th>
<th>Angkatan</th><th>Aksi</th></tr>
    </thead>
    <tbody>
        <?php foreach ($daftar as $i => $m): ?>
            <tr>
                <td><?= $i + 1 ?></td>
                <td><?= e($m->getNim()) ?></td>
                <td><?= e($m->getNama()) ?></td>
                <td><?= e($m->getEmail()) ?></td>
                <td><?= e($m->getProdiNama()) ?></td>
                <td><?= e($m->getAngkatan()) ?></td>
                <td>
                    <a href="<?= url('/mahasiswa/edit?id=' . $m->getId())
?>" class="btn btn-sm btn-warning">Edit</a>
                    <form method="POST" action="<?=
url('/mahasiswa/destroy') ?>" class="d-inline"
                          onsubmit="return confirm('Yakin ingin menghapus
data ini?');">
                        <input type="hidden" name="id" value="<?= $m-
>getId() ?>">
                        <button type="submit" class="btn btn-sm btn-
danger">Hapus</button>
                    </form>
                </td>
            </tr>
        <?php endforeach; ?>
        <?php if (!$daftar): ?>
            <tr><td colspan="7" class="text-center">
```

```
                <?= $keyword !== '' ? 'Tidak ada data yang cocok.' : 'Belum
ada data.' ?>
            </td></tr>
        <?php endif; ?>
    </tbody>
</table>

<?php require __DIR__ . '/../footer.php'; ?>
```

app/Views/auth/login.php

```
<?php
$flash = getFlash();   // flash message: tampil sekali lalu hilang
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Login - SI Akademik</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body class="bg-light">

<div class="container" style="max-width: 420px; margin-top: 10vh;">
    <div class="card shadow-sm">
        <div class="card-body p-4">
            <h1 class="h4 mb-3 text-center">SI Akademik</h1>

            <?php if ($flash): ?>
                <div class="alert alert-<?= e($flash['type']) ?>"><?=
e($flash['message']) ?></div>
            <?php endif; ?>

            <form method="POST" action="<?= url('/login') ?>">
                <div class="mb-3">
                    <label class="form-label">Username</label>
                    <input type="text" name="username" class="form-control"
required autofocus>
                </div>
                <div class="mb-3">
                    <label class="form-label">Password</label>
                    <input type="password" name="password" class="form-
control" required>
                </div>
                <button type="submit" class="btn btn-primary w-
100">Login</button>
```

```
            </form>
        </div>
    </div>
</div>

</body>
</html>
```

BAB V HASIL DAN PEMBAHASAN

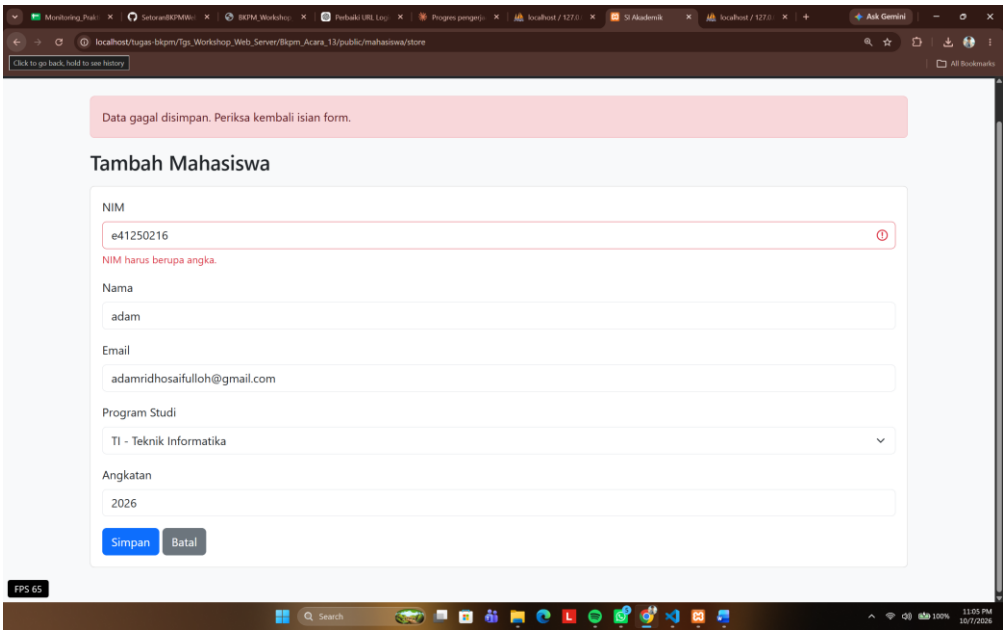
5.1 Hasil Implementasi

Berdasarkan source code Acara 13, Service Layer telah diterapkan melalui MahasiswaService. Controller menjadi lebih sederhana karena tidak lagi menangani validasi maupun query database. MahasiswaRepository dan ProdiRepository bertanggung jawab terhadap akses data, sedangkan MahasiswaService menangani validasi dan business logic.

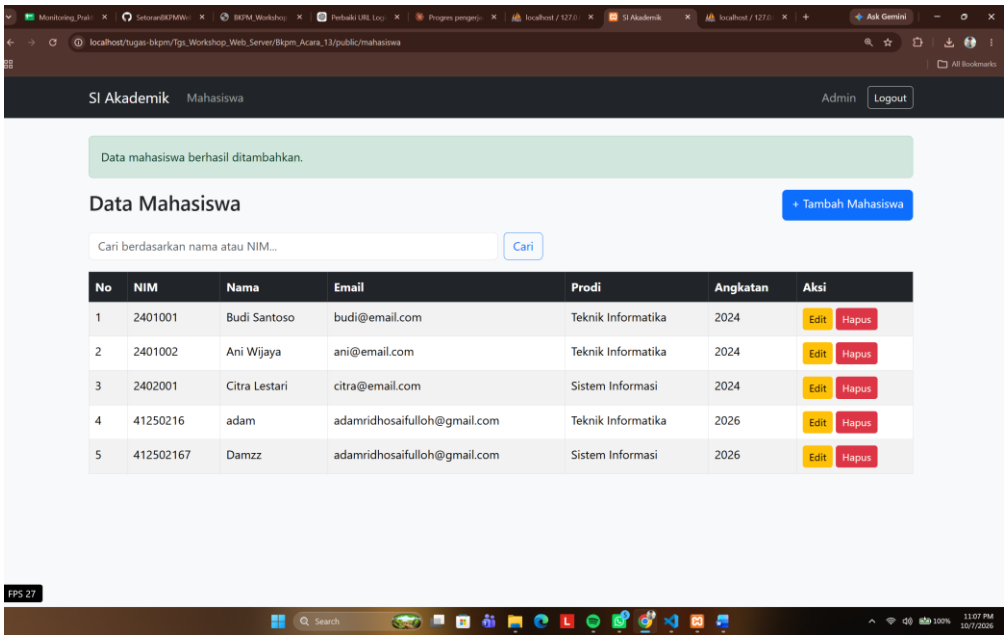
5.2 Flash Message

Flash Message digunakan untuk menampilkan informasi proses kepada pengguna. Pesan keberhasilan ditampilkan setelah data berhasil ditambahkan, diubah, atau dihapus. Pesan kegagalan digunakan untuk validasi seperti NIM sudah terdaftar atau data tidak valid. Pesan hanya ditampilkan satu kali karena setelah diambil melalui getFlash(), data dihapus dari session.

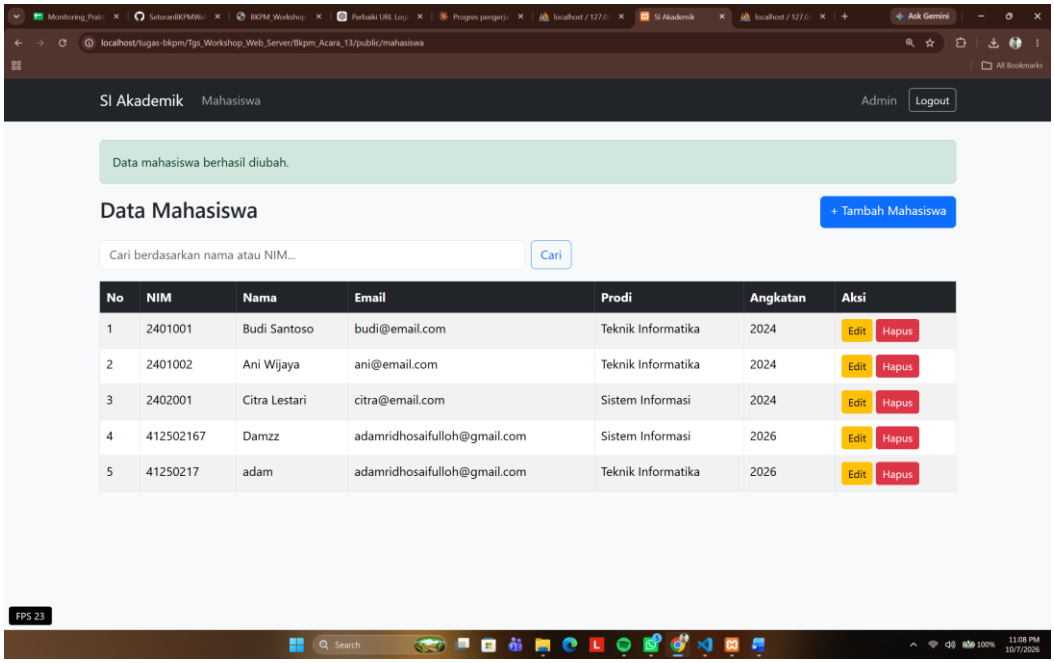
1. **Data Gagagl Disimpan**



2. Data Berhasil Ditambahakan



3. Data Berhasil Diiubah



5.3 Pengujian

Pengujian	Skenario	Hasil
Tampil data mahasiswa	Membuka /mahasiswa	Daftar mahasiswa tampil.
Pencarian	Mengisi keyword nama/NIM	Data yang sesuai keyword tampil.
Tambah valid	Mengisi form dengan data valid	Data tersimpan dan muncul pesan berhasil.
NIM kosong	Mengosongkan NIM	Validasi NIM tampil.
NIM duplikat	Menggunakan NIM yang sudah ada	Pesan NIM sudah terdaftar tampil.
Email tidak valid	Mengisi format email salah	Pesan format email tidak valid tampil.
Ubah data	Mengedit data mahasiswa	Data berubah dan pesan berhasil tampil.
Hapus data	Menekan tombol Hapus	Data terhapus dan pesan berhasil tampil.

BAB VI KESIMPULAN

Pada Acara 13 telah diterapkan Service Layer, validasi, session, dan business logic pada aplikasi Sistem Informasi Akademik. MahasiswaService menjadi penghubung antara Controller dan Repository serta menangani validasi dan aturan bisnis. Dengan pembagian tersebut, Controller menjadi lebih sederhana, akses database tetap berada di Repository, dan Flash Message dapat memberikan informasi proses kepada pengguna secara terstruktur.